



Sandya Mannarswamy

Welcome to another instalment of CodeSport. Here, we discuss the solutions to a few of the questions on multi-threaded programming that we had featured in last month's column.

Many readers had sent their answers to the multi-threaded programming questions. Congratulations to Satish Kumar and Vivekandam for getting the maximum number of answers correct. In this month's column, we will discuss the solutions to a few of the questions from the previous issue of *LFY*.

Composability of locks

Our first question was about the composability of locks. One of the main problems associated with locks is that they are not composable. We had discussed this issue in one of our earlier columns as well. Composability implies that, given two operations A and B, which are correct individually, it is possible to compose them together into a single operation that will automatically obey all correctness properties. For instance, a function 'foo', which acquires Lock X and Y, in that order, cannot be integrated into a system containing a function 'bar' that acquires Locks Y and X, in that order, if the two functions 'foo' and 'bar' are concurrently executable. Such a composition can lead to a deadlock.

As we had discussed earlier, deadlocks can be avoided by acquiring locks in a predetermined static order. However, this requires that all programmers developing an application should be aware of the static lock acquiring order and should follow the order consistently. Enforcing static lock ordering becomes cumbersome as more modules get added.

Currently, there is no way to express the set of locks acquired by a function as part

of the function signature in a programming language like C. This prevents easily checking whether a function obeys the static lock order or not. Checking the static lock order to avoid deadlocks requires whole program inter-procedural analysis by the compilers/tools, which is difficult and time consuming. For a discussion on the various issues caused by lock-based programming, you should look up the seminal article 'Problem with Threads' by Edward Lee available at, <http://ptolemy.eecs.berkeley.edu/publications/papers/06/problemwithThreads>.

Because of the lack of composability using lock-based programming, many alternatives have been proposed, such as lock-free data structures and transactional memory. We had discussed lock-free data structures and transactional memory in one of our earlier columns. However, there is no silver bullet to avoid the issues with lock-based programming. While the alternatives, such as lock-free data structures and software transactional memory, are interesting from a research perspective, none of them have gained enough popular acceptance to be a viable alternative to traditional lock-based programming.

Reader/writer locks

Our second question from last month's column was on reader/writer locks. When would you prefer using reader/writer locks over simple mutual exclusion locks? If you have a data structure that often has concurrent read operations being performed on it and only has infrequent update operations, then a reader-